Curated Antigravity Prompts for VYBIN Phases 2 & 3

This document contains comprehensive, sequential prompts to guide the Antigravity AI through Phase 2 (Local Logic & Cryptography) and Phase 3 (Firebase Integration) of the VYBIN messaging app. Feed these to the AI one by one, verifying the output before proceeding to the next.

Phase 2: Semantic Development (Local Logic & Cryptographic Engine)

Phase 2 Dependencies: Ensure these are in `pubspec.yaml` before starting this phase.

- `flutter_bloc: ^8.1.3` (or latest)
- `equatable: ^2.0.5`
- `pointycastle: ^3.7.3`
- `flutter_secure_storage: ^9.0.0`
- `image_picker: ^1.0.4`
- `record: ^5.0.5`
- `just_audio: ^0.9.34`
- `flutter_image_compress: ^2.1.0`
- `go_router: ^14.0.0` (for navigation routing)

Prompt 2.1: State Management & Routing Core

```
Action: Set up core BLoC architecture, State Management, and GoRouter navigation

Context:
We are beginning Phase 2. The UI is built, but we need to decouple state from the

Task:
1. Setup GoRouter in `lib/app.dart` or `lib/core/routes.dart`. Define routes for:
2. Ensure the Route Guard checks auth state (stub it for now: if logged out -> Lo
3. Create the `AuthBloc` (auth_bloc.dart, auth_event.dart, auth_state.dart) using
4. Create the `ChatBloc` for handling 1-on-1 chats and `ChatListBloc` for the inb
5. Wrap the root `MaterialApp` with `MultiBlocProvider` injecting these BLoCs so
```

Prompt 2.2: Cryptographic Engine & Secure Storage

```
Action: Implement the localized Cryptographic Engine and Secure Key Storage.

Context:
VYBIN is an E2EE chat app. We need a robust, local cryptographic service utilizin

Task:
1. Create `core/services/secure_key_storage.dart`. Use `flutter_secure_storage` t
2. Create `core/services/encryption_service.dart`. Implement the following precis
```

```
   - `generateKeyPair()`: Generates an RSA-2048 key pair. (Note: Run this in a Da
   - `deriveKeyFromPassword(String password, String uid)`: Uses PBKDF2-HMAC-SHA25
   - `encryptPrivateKey(RSAPrivateKey privKey, Uint8List derivedKey)`: Encrypts t
   - `decryptPrivateKey(String encryptedPrivKeyB64, Uint8List derivedKey)`: Decry
   - `clearPrivateKey()`: Nullifies the in-memory private key for logout.
3. Write localized unit tests (or setup simple mock tests) to validate that encry
```

Prompt 2.3: Local Media Processing Services

```
Action: Implement device hardware integration for Media capturing and processing.

Context:
Users will send images and voice notes. We need a local `MediaService` to interfa

Task:
1. Create `core/services/media_service.dart`.
2. Implement image selection using `image_picker`. Include a method to compress t
3. Implement audio recording using the `record` package. Provide methods to `star
4. Implement basic playback logic using `just_audio` to play a given local audio
5. Ensure proper Android/iOS permission requests (Camera, Microphone, Storage) ar
```

Phase 3: Firebase Integration (Live Sync & DB Governance)

Phase 3 Dependencies: Ensure these are added and `flutterfire configure` has been run.

- `firebase_core: ^3.0.0`
- `firebase_auth: ^5.0.0`
- `cloud_firestore: ^5.0.0`
- `firebase_storage: ^12.0.0`
- `firebase_messaging: ^15.0.0`

Prompt 3.1: Authentication & Username Registry

```
Action: Integrate Firebase Auth and the atomic Username Registry flow.

Context:
We are connecting our UI and AuthBloc to Firebase. VYBIN uses email/password auth

Task:
1. Create `features/auth/data/auth_repository.dart`.
2. Implement Login, Signup, and Logout methods wrapping Firebase Auth.
3. **CRITICAL SIGNUP FLOW**:
   - Before calling Firebase Auth create user, query the Firestore `usernames` co
   - On Firebase Auth success, generate the RSA key pair via `EncryptionService`.
   - Derive the key from the password, encrypt the private key, and save it via `
   - Write to Firestore in a batch/transaction:
     a) `users/{uid}`: uid, username, displayName, email, publicKey (PEM string),
     b) `usernames/{username}`: uid (for exact match lookups).
```

```
4. Connect `AuthRepository` to `AuthBloc`. On login success, re-derive the AES ke
```

Prompt 3.2: Chat Repository & Real-time Synchronization

```
Action: Implement Firestore real-time queries for New Chats and the Inbox.

Context:
Users need to find each other via exact username search, initiate conversations,

Task:
1. Create `features/chat/data/chat_repository.dart`.
2. Implement User Search: Query `usernames/{input}` for an exact match. If found,
3. Implement Conversation ID generation: Given `uid1` and `uid2`, sort them alpha
4. Implement `createConversation()`: If a chat doesn't exist, create a document i
5. Implement a real-time stream `getConversationsStream()` that listens to the `c
6. Connect this stream to `ChatListBloc` so the Home screen updates instantly whe
```

Prompt 3.3: End-to-End Encryption Messaging Pipeline

```
Action: Integrate the core E2EE message sending and receiving pipeline.

Context:
This is the heart of VYBIN. Messages must be encrypted with AES-256-GCM. The AES

Task:
1. In `EncryptionService`, implement `encryptMessage(String plaintext, String rec
   - Generate a random 32-byte AES session key and 12-byte IV.
   - Encrypt the plaintext.
   - RSA-OAEP encrypt the session key twice (once for recipient, once for sender)
   - Return an object with: ciphertext, iv, and the two encrypted keys.
2. In `EncryptionService`, implement `decryptMessage(EncryptedMessage msg)`:
   - Use the in-memory private key to decrypt the AES session key, then decrypt t
3. In `ChatRepository`, implement `sendMessage()`:
   - Fetch recipient's public key.
   - Encrypt the text.
   - Write to `conversations/{id}/messages/{messageId}` with fields: `senderUid`,
   - Update the parent conversation document's `lastMessagePreview` (with the sam
4. Implement `getMessagesStream(conversationId)`: Listen to the messages subcolle
```

Prompt 3.4: Secure Media Storage & Push Notifications

```
Action: Implement Encrypted Media Uploads and Push Notification integration.

Context:
Media files must also be encrypted before touching Firebase Storage. We also need

Task:
```

```
1. In `ChatRepository`, implement `sendMediaMessage()` (for images/audio):
   - Get bytes from `MediaService`.
   - Generate an AES session key and encrypt the raw bytes.
   - Upload the ENCRYPTED bytes to Firebase Storage `media/{conversationId}/{mess
   - Get the download URL.
   - Encrypt the session key with RSA-OAEP for both participants.
   - Save the message to Firestore with `type: image/voice`, `mediaUrl`, and the
2. Implement Firebase Cloud Messaging (FCM) setup in `main.dart`. Request permiss
3. Handle background FCM taps using `FirebaseMessaging.onMessageOpenedApp`, routi
```